

Architectural Design Decisions for Monitoring Deployed Reinforcement Learning Systems

Zhizhou Fang^{1,2}[0000–0002–7013–8182], Stephen John Warnett^{1,2}[0000–0003–0650–0981], and Uwe Zdun²[0000–0002–6233–2591]

¹ Doctoral School Computer Science (DoCS), University of Vienna, Austria

² Research Group Software Architecture, Faculty of Computer Science, University of Vienna, Austria

`firstname.lastname@univie.ac.at`

`https://swa.cs.univie.ac.at`

Abstract. Reinforcement learning (RL) systems deployed in production face monitoring challenges absent in traditional machine learning: policies degrade silently under environment drift, reward signals can be gamed, and agents may develop hacking behaviors that evade standard evaluation. Despite growing practitioner interest, no systematic study has mapped the architectural design decisions (ADDs) practitioners face when building monitoring infrastructure for deployed RL agents. We address this gap through a Straussian Grounded Theory analysis of 29 gray literature sources, performing open, axial, and selective coding to extract ADDs, decision options, and decision drivers related to post-deployment RL monitoring. Our analysis identifies seven ADDs organized along a layered monitoring pipeline, spanning signal selection, statistical degradation testing, false alarm calibration, automated response mechanisms, and reward hacking detection and mitigation, together with 57 decision options and 72 decision drivers grounded in verbatim evidence. The emerging theory reveals that monitoring must be multi-signal and adversary-aware, as aggregate metrics can remain stable while dangerous behaviors grow in specific interaction contexts. The results are formalized as a decision model with UML visualizations to support practitioners in navigating the RL monitoring design space, with the full catalog available in our replication package.

Keywords: reinforcement learning · deployment monitoring · software architecture · architectural design decisions · grounded theory · gray literature · MLOps.

1 Introduction

Reinforcement learning (RL) has moved from research experiments to production systems that make consequential decisions, from recommendation engines and dynamic pricing to robotic control and dialogue agents. A deployed RL agent interacts continuously with its environment, and its performance depends

on the stability of the environment’s dynamics, reward structure, and state distribution. When any of these shift after deployment, a phenomenon variously called environment drift, concept drift, or distribution shift, the agent’s policy can degrade silently, sometimes catastrophically, without any change to the policy itself [15] [9] [10].

This factor makes monitoring deployed RL systems both critically important and qualitatively different from monitoring supervised machine learning models. Supervised ML monitoring techniques, such as input-distribution tracking and accuracy dashboards, address only part of the problem. RL monitoring must additionally contend with delayed and sparse reward signals, non-stationary environments, the possibility of reward hacking, where an agent maximizes a proxy reward while violating the designer’s true intent, and the risk that an agent learns to evade its own monitors [1] [30]. These RL-specific failure modes demand dedicated architectural decisions about what to monitor, how to detect degradation statistically, how to distinguish genuine performance loss from reward manipulation, and how to respond when problems are detected.

Despite growing practitioner interest, evidenced by a rising volume of engineering blogs, industry reports, and applied research publications on RL deployment, no systematic study has mapped the architectural design decisions (ADDs) that practitioners and researchers face when building monitoring infrastructure for deployed RL agents. Prior ADD studies for ML cover the upstream workflow: data processing, feature persistence, model building, and AutoML [35], and the deployment layer: CI/CD automation, pipeline triggering, data versioning, model versioning, and MLOps adoption [36]. These studies acknowledge monitoring as a prerequisite practice: model performance degradation and data distribution changes appear as pipeline trigger options, and model performance monitoring is noted as a required practice for the degradation trigger [36]. However, monitoring itself is never decomposed into its own architectural decisions. The questions of what signals to monitor, which statistical tests to apply, how to calibrate detection thresholds, and how to respond when degradation is detected remain outside the scope of these models.

A recent multivocal literature review of 136 ML monitoring studies [22] comprehensively catalogs monitoring techniques, metrics, and tools for production ML systems, but confirms that RL monitoring is rare. Only five of 136 reviewed studies address RL, all in formal literature, and no existing work organizes monitoring practices into ADDs. The gap is even wider for the other monitoring challenges in RL systems, where the monitoring challenges described above (environment non-stationarity, reward hacking, and monitor evasion) have no counterpart in the general ML monitoring literature. Similarly, while the academic literature proposes specific detection algorithms for individual RL monitoring concerns, such as environment drifts [10] [9], reward degradation [13], and agents’ behaviors [39], each addresses a single monitoring concern in isolation. Existing work does not organize these diverse detection methods into a coherent decision space that a practitioner could navigate when designing a monitoring architecture for a deployed RL system.

We address this gap through a Straussian Grounded Theory (GT) analysis of gray literature on RL deployment monitoring and drift detection. Gray literature, such as engineering blogs, practitioner guides, conference talks, GitHub repositories, and informal reports, was chosen as the primary data source because it most directly represents the voice and contemporary understanding of practitioners [12] [26]. Following the methodology of Strauss and Corbin [6] [5], we iteratively collected sources, performed open, axial, and selective coding with constant comparison and memoing, and modeled the emerging theory as a formal decision model with UML visualizations. We coded 29 sources to theoretical saturation, applying the same research method successfully used in prior ADD studies for the ML workflow [35] and ML deployment [36].

Our study addresses the following research questions:

- **RQ1:** Which ADDs are available to choose from in the context of monitoring deployed RL systems, and what are the relations between them?
- **RQ2:** What are the relations between these decisions and their decision options?
- **RQ3:** Which decision drivers (forces) are relevant to the decision options?

By structuring the results as a formal model, the study maps the relationships among ADDs, decision options, and decision drivers in the context of monitoring deployed RL systems. This framework not only offers a practical roadmap for industry professionals but also advances academic understanding of the field’s methodologies and challenges.

The remainder of this paper is structured as follows: Section 2 discusses related work, Section 3 describes the research method, Section 4 presents the ADDs, Section 5 interprets the results and considers threats to validity, Section 6 concludes, and Section 7 describes our replication package availability.

2 Related Works

Our study is related to the following research areas: ADD modeling for ML systems, runtime monitoring of production ML systems, and RL-specific monitoring. This section discusses the related works of each area.

2.1 Architectural Design Decisions for ML Systems

The growing adoption of ML in production systems has prompted research into the software engineering (SE) and architectural challenges practitioners face. Sculley et al. [28] identified ML-specific sources of technical debt, including entanglement, hidden feedback loops, data dependencies, and configuration issues that make production ML systems costly to maintain. Washizaki et al. [37] categorized software patterns and smells for ML pipelines. Lwakatare et al. [19] proposed an empirical taxonomy of SE challenges for ML systems. Bosch et al. [3] identified open research areas at a high level of abstraction. Nascimento

et al. [21] conducted a comprehensive SLR on SE for AI and ML, and Serban et al. [29] surveyed adoption of engineering best practices. In prior work [35,36], we modeled ADDs for the ML workflow and deployment using Straussian GT on gray literature, producing UML-based decision models with decision options and drivers. Ntentos et al. [23] extended this line to RL training strategies. Collectively, these studies cover SE challenges, patterns, and ADDs for ML data processing, model building, deployment, and serving. None of these works decomposes monitoring into its own ADDs, nor addresses the RL-specific failure modes of environment drift, reward hacking, and monitor evasion. Our aforementioned prior studies [35,36], the closest methodological predecessors, treat monitoring only as a pipeline trigger rather than a structured decision space. In contrast, the study described herein directly extends our existing ADD landscape into this gap, applying the same GT approach to a domain that those works acknowledge but leave unmodeled.

2.2 Monitoring Production ML Systems

A growing body of work addresses the challenge of monitoring ML systems after deployment. Schröder and Schulz [27] proposed a taxonomy of ML monitoring methods across data, model, and non-technical categories. Karval and Singh [16] surveyed drift detection, outlier detection, and adversarial detection techniques. Leest et al. [18] classified contextual information in ML monitoring along system, aspect, and representation dimensions. Protschky et al. [25] combined a multi-vocal literature review with practitioner interviews, finding that practitioners struggle with monitoring configuration and that existing tools do not support the full monitoring lifecycle. In the domain of safety-critical systems, Ferreira et al. [11] surveyed safety monitoring for ML perception functions in cyber-physical systems but did not extend their scope to RL agents. Most comprehensively, Naveed et al. [22] reviewed 136 monitoring studies, confirming that supervised learning dominates the literature and that RL monitoring is rare, with only five of 136 studies addressing it. The production ML monitoring literature targets stationary supervised models and does not account for the reward signal, sequential decision-making, or adversarial failure modes specific to RL. Protschky et al. [25] come closest to a practitioner-facing framework but do not organize their findings as ADDs. None of these works provides decision options, drivers, or trade-offs structured for RL deployment. Our study fills this gap by applying the ADD modeling paradigm to RL monitoring, producing the structured decision model that these surveys confirm is absent.

2.3 RL-Specific Monitoring

Research on monitoring deployed RL systems addresses three distinct concerns: detecting environment drifts, detecting unsafe agent behaviors at runtime, and detecting reward hacking, where the agent exploits its objective function. These three threads have developed largely independently, each proposing individual detection or mitigation algorithms without considering how they interact as

ADDs within a deployed monitoring pipeline. Our study addresses this gap by identifying and organizing the ADDs that span all three concerns.

Environment Drift Detection Environment drift in RL refers to changes in the underlying transition dynamics $P(o_{t+1}|o_t, a_t)$ that can silently degrade a deployed agent’s policy performance. Wang et al. [34] proposed detecting environment drift by exploring the environment with equal probability. Greenberg and Mannor [13] applied a statistical test over the episodic returns. Fang and Zdun [9, 10] used Gaussian Processes and SVMs, respectively. Haider et al. [15] developed PEDM using probabilistic ensemble models. Danesh and Fern [7] applied recurrent implicit quantile networks, and Wang et al. [33] proposed OIL-AD, a transformer-based anomaly detection framework.

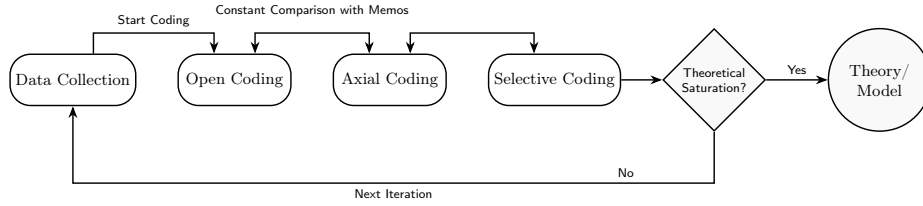
Agent Safety Monitoring Mallozzi et al. [20] introduced a framework that blocks unsafe actions and uses reward shaping to penalize violations. Chen et al. [4] proposed a runtime assurance framework for autonomous vehicles that switches between an unverified RL controller and a verified safe fallback. Zolfagharian et al [39] presented a black-box approach that predicts safety violations from an agent’s Q-values.

Reward hacking and alignment monitoring Skalse et al. [30] provided the first formal definition of reward hacking. Pan et al. [24] conducted the first systematic empirical study across four RL environments. Laidlaw et al. [17] proposed occupancy-measure regularization as a mitigation. Greenblatt et al. [14] demonstrated that frontier language models can strategically fake alignment under monitoring. Baker et al. [2] showed that chain-of-thought monitoring can detect reward hacking in frontier reasoning models.

3 Research Method

To explore how industry practitioners conceptualize the monitoring of RL agents in production, we conducted a systematic gray literature study (GLS) [12] grounded in Straussian Grounded Theory (GT) [6] [5]. GT is a qualitative, inductive research method in which theory emerges from systematic coding and constant comparison of empirical data, with data analysis occurring simultaneously with data collection rather than afterward. Gray literature, defined as any material not formally peer-reviewed nor formally published [12], such as blog posts, practitioner guides, and industry reports, was chosen as the primary data source because it most directly represents the voice and contemporary understanding of practitioners [26].

Figure 1 illustrates our research method. We searched for practitioner and applied research sources using standard search engines (e.g., Google, DuckDuckGo) and topic portals (e.g., Towards Data Science, Medium). The initial search term was “reinforcement learning in production”. Subsequent sources were searched

**Fig. 1.** Research Method

for using terms guided by theoretical sampling, including “RL production monitoring”, “environment drift detection RL”, “reward hacking detection”, “RL safety dashboards”, “RLHF deployment”, and “RL retraining playbooks”. We then repeatedly and iteratively applied GT coding practices and constant comparison to identify concepts, categories, properties, and relations. The resulting entities were modeled in Python code, which was then used to generate a UML model. Subsequent sources were actively sought based on topics identified in the previous iteration, with particular consideration given to topics needing coding and their potential contribution to the model.

Sources were deemed candidates if they were relevant to the monitoring of deployed RL systems, i.e., they concerned observing, evaluating, or maintaining agent behavior after training is complete and the policy is deployed. Sources focused exclusively on RL training, algorithm design, or general ML deployment without RL-specific monitoring content were excluded.

Our methodology began with open coding, where conceptual details were systematically translated into distinct labels. This process was followed by axial coding, grouping recurring and synonymous concepts into broader categories. Throughout these initial stages, we conducted an in-depth analysis of each source, documenting our interpretations and reasoning in memos to ensure full traceability between the data and the resulting codes. During the selective coding phase, we synthesized these findings to identify core theoretical themes, revisiting previous data as needed to refine our understanding. To conclude, we integrated formal modeling with UML generation into the axial and selective stages, ensuring the final theory was both precise and internally consistent.

Sources were classified into three categories based on their contribution to the emerging theory. Primary sources introduced new ADDs and were coded in full with detailed memos. Supporting sources did not introduce new ADDs but contributed new decision options or decision drivers to existing ADDs. Confirming sources confirmed existing ADDs, decision options, and decision drivers without adding new elements.

Following Strauss and Corbin [5], we considered theoretical saturation to be reached when subsequent sources no longer introduced new ADDs, decision options, or decision drivers. In our study, the last eleven sources confirmed existing categories without substantively altering the decision structure. Only one source (s26) contributed a single additional option to an existing ADD with

nine options, confirming the stability of the overall model. In total, we coded 29 sources to saturation, identifying seven ADDs, 53 decision options, and 59 decision drivers. Our knowledge sources are summarized in Table 1.

We utilized CodeableModels³ to model the phenomena we discovered. This Python-based framework then facilitated the automated generation of UML diagrams through integrated PlantUML⁴ model view generators. Project artifacts can be found in our replication package [8].

Table 1. List of Knowledge Sources Included in the Study.

ID	Title	Source Type	Class
s1	RL in Production: Building Adaptive AI Systems	Practitioner Article	Primary
s2	Detecting Rewards Deterioration in RL	Practitioner Article	Primary
s3	Building a Self-Healing RL Agent	Practitioner Article	Supporting
s4	What is Offline RL? (2026 Guide)	Practitioner Article	Supporting
s5	Closing the Eval-Deployment Gap in AI Systems	Practitioner Article	Supporting
s6	Reward Hacking: Detection and Prevention (2026)	Practitioner Article	Primary
s7	Implementing Automated Rollback Mechanisms	Practitioner Article	Supporting
s8	Handling LLM Model Drift in Production	Practitioner Article	Supporting
s9	Deployment Considerations for RLHF Models	Practitioner Article	Supporting
s10	RLaaS: RL at Scale for Enterprise	Practitioner Article	Confirming
s11	Reward Hacking in RL (Lilian Weng)	Blog Post	Supporting
s12	Reward Hacking in RL (Pan et al. summary)	Topic Summary	Confirming
s13	Emergent Misalignment from Reward Hacking	Research Blog	Supporting
s14	Reward Hacking in RLVR Systems	Topic Summary	Supporting
s15	OOD Detection for RL (Fraunhofer IKS)	Research Blog	Confirming
s16	When RL Safety Constraints Break	Practitioner Article	Supporting
s17	How Can RL Agents Be Monitored?	Q&A Forum	Confirming
s18	Diagnosing Instability in Production RL	Engineering Blog	Supporting
s19	Deploying RL Models: MLOps Challenges	Practitioner Article	Confirming
s20	Deploying RL with Ray and Amazon SageMaker	Engineering Blog	Confirming
s21	Deploying RL Agents in Private Clouds	Practitioner Article	Confirming
s22	RLHF at Scale: Enterprise LLMs with HITL	Practitioner Article	Confirming
s23	RLHF: Enterprise Integration	Practitioner Article	Confirming
s24	Evaluate, Monitor, Tune LLMs	Practitioner Article	Confirming
s25	Reward Hacking Is Becoming More Sophisticated	Forum Post	Confirming
s26	MONA: Steganography Without Optimisation	Forum Post	Supporting
s27	CoT Monitoring for Frontier Reasoning Models	Research Blog	Confirming
s28	Reasoning Models Don't Always Say What They Think	Research Blog	Confirming
s29	Gaming the System: Reward Hacking in LLM Training	Blog Post	Confirming

4 Architectural Design Decisions

We present the ADDs derived from the gray literature in this section. An overview of the various ADDs, their corresponding number of decision options, and the number of decision drivers (forces) is provided in Table 2. The full catalog of decision options, decision drivers, and gray literature sources is available in our replication package [8]. Our analysis identified seven ADDs organized along a layered monitoring pipeline that addresses two distinct failure axes: reward deterioration, where the agent's performance degrades due to environment drift,

³ <https://github.com/uzdun/CodeableModels>

⁴ <https://plantuml.com>

and **reward hacking**, where the agent’s measured reward remains high while its true performance degrades. Figure 2 illustrates the seven ADDs and the relations between them. ADD 1 addresses **safe exploration** in production. ADDs 2–5 form the **detection layer**: ADD 2 determines which signals to monitor, ADD 3 selects how to respond when issues are detected, ADD 4 chooses the statistical test for reward degradation, and ADD 5 calibrates false alarm thresholds. ADDs 6–7 address the **reward hacking** threat: ADD 6 selects a detection strategy, and ADD 7 selects a mitigation architecture. Throughout this section, individual decision options are written in ***CamelCase bold italics*** (e.g., ***ConservativePolicyUpdates***), decision drivers are written in *CamelCase italics* (e.g., *SafetyConstraints*), and analytical groupings of related options into families are written in *italic title case* (e.g., *Agent-Internal Signals*).

Table 2. Overview of ADDs, the Number of Decision Options and the Number of Decision Drivers.

ADD ID	ADD Name	#Decision Options	#Decision Drivers
1	Safe Exploration Strategy Selection	3	4
2	Production Monitoring Signal Selection	16	17
3	Automated Response Mechanism Selection	9	12
4	Reward Degradation Test Statistic Selection	6	9
5	False Alarm Rate Control Method	4	5
6	Reward Hacking Detection Strategy	9	16
7	Reward Hacking Mitigation	10	9

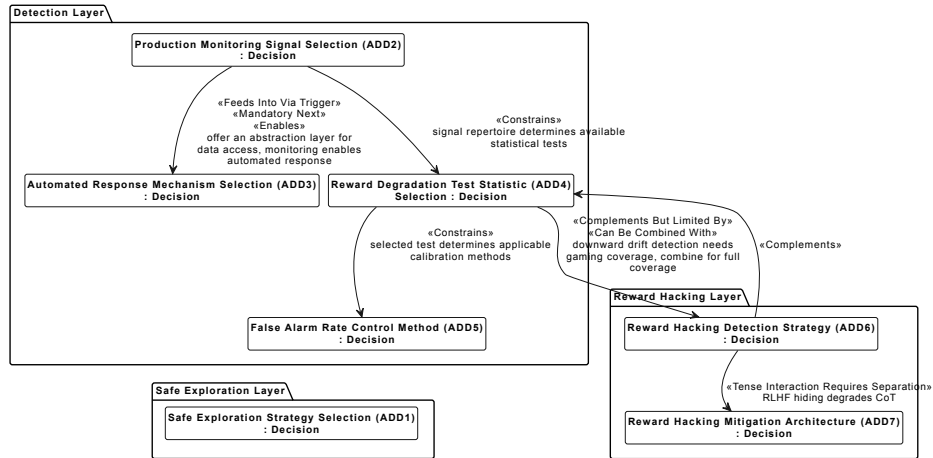


Fig. 2. Overview of the seven ADDs for monitoring deployed RL systems.

4.1 Safe Exploration Layer

ADD 1 addresses the fundamental tension in a production RL system: how much freedom should the agent explore during deployment? **Exploration is necessary for the agent to adapt to a non-stationary environment.** Yet, unconstrained exploration in production can cause direct harm, violate operational constraints, or produce catastrophic performance regressions that are difficult to recover from [32]. ADD 1 and the monitoring layer are mutually dependent: safe exploration requires monitoring to be in place, since circuit breakers and performance signals are what make bounded policy updates recoverable when they go wrong. The decision is therefore not whether to explore, but how to bound it. ADD 1 yields 3 decision options and 4 decision drivers.

The entire option space is shaped by two forces that constrain it from the outset: *SafetyConstraints* and *CatastrophicDegradation* together rule out unconstrained exploration in production, establishing a shared baseline requirement that any chosen strategy must limit what the agent can do and how fast it can change. Within that constrained space, *EnvironmentNonStationarity* and *DelayedSparseRewards* act as the differentiating forces that pull practitioners toward one option over another. ***ConservativePolicyUpdates***, which limit the magnitude of each policy change to prevent large deviations from a validated baseline, are the most direct expression of *SafetyConstraints* and *CatastrophicDegradation*. Still, they respond poorly to *EnvironmentNonStationarity*: a globally penalized deviation budget leaves little room to track a drifting environment. ***ConstrainedPolicyOptimization***, which encodes safety requirements directly into the optimization objective through constrained MDPs or safe-RL formulations, resolves this tension by separating the safety envelope from the exploration range, allowing the agent to adapt freely within bounds; it is therefore favored when *EnvironmentNonStationarity* is high, but *SafetyConstraints* remain strict. ***MultiArmedBanditIntegration***, which routes a fraction of traffic to candidate policy variants and treats each as a bandit arm, is the option most distinctively shaped by *DelayedSparseRewards*: when reward feedback is infrequent or noisy, structured variant testing is statistically more reliable than free exploration, and traffic splitting addresses *CatastrophicDegradation* by limiting live exposure to any single untested policy.

4.2 Detection Layer

ADDs 2–5 form the detection layer that, together, operationalize monitoring in production. ADD 2 determines which signals to observe, establishing the informational foundation on which all subsequent decisions rest. ADD 3 selects how the system should respond when those signals cross a threshold, coupling detection directly to intervention. ADD 4 chooses the statistical test used to identify reward degradation from the observed signals, and ADD 5 calibrates the false alarm rate of that test. The four decisions are therefore not independent: the signal repertoire chosen in ADD 2 constrains the statistical tests available in ADD 4, since some tests assume episodic structure or specific distributional

properties that not all signals satisfy; the chosen test in ADD 4, in turn, determines what calibration methods are applicable in ADD 5. Furthermore, as shown in Figure 2, ADD 4 also feeds into ADD 6 via a complementary relationship: downward drift detection addresses reward deterioration, while reward hacking requires gaming coverage, and the two must be combined for full coverage across both failure axes. A central cross-cutting tension runs through all four decisions: broader and more diverse signal coverage improves detection sensitivity across both failure axes, but increases operational complexity, cost, and the risk of alert fatigue, which ADD 5 specifically exists to manage.

ADDs 2–5 collectively yield 35 decision options and 43 decision drivers; the full catalog is available in our replication package [8]. The driver landscape across ADDs 2–5 is shaped by three dominant forces that constrain the entire option space. *EnvironmentNonStationarity* and *SignalNonStationarity* together establish that static, single-metric approaches are insufficient: the environment and the signals used to observe it both change over time, disfavoring fixed thresholds and naive moving averages in favor of adaptive and multi-signal strategies. *CatastrophicDegradation* and *FlappingRisk* exert opposing pressure across ADD 3 and ADD 5, the former demanding fast, decisive responses and the latter penalizing overly sensitive thresholds that trigger unnecessary rollbacks. Within that constrained space, the differentiating forces resolve into three option families. The first family, *Agent-Internal Signals*, including ***RLSpecificMetricTracking***, ***CovarianceWeightedRewardSignal***, ***RewardModelScoreMonitoring***, and ***KLDivergenceFromReferencePolicy***, is driven primarily by *RewardModelStaleness*, *AlignmentImpermanence*, and *EvaluationMismatch*, and pairs naturally with ***CovarianceWeightedMean-Test*** in ADD 4 and ***BootstrapFAR*** in ADD 5. The second family, *Environment-Facing Signals*, including ***FeatureDistributionDrift***, ***EmbeddingDriftDetection***, and ***ActivationDeltaMonitoring***, is driven by *CovariateShift*, *HiddenVariableShift*, and *DriftInevitability*, and motivates ***AdaptiveThreshold*** in ADD 5 since distributional shift invalidates fixed baselines. The third family, *Outcome-Facing Signals*, including ***BusinessOutcomeCorrelation***, ***Reward-ProxyGapTracking***, ***ShieldInterventionRateTracking***, and ***ActionFailureRateMonitoring***, is driven by *GoodhartsLaw*, *MonitoringIndependence*, and *DelayedSparseRewards*, and connects most directly to ADD 3, where ***AutomatedCircuitBreakers*** and ***HumanInTheLoopIntervention*** are triggered by business-level rather than internal metrics. *GovernanceTraceability* and *SecurityCompliance* apply across all three families, favoring signals and response mechanisms whose baselines, thresholds, and audit trails can be versioned and explained. This consideration bears on both the choice of statistical test in ADD 4 and the response architecture in ADD 3.

4.3 Reward Hacking Layer

ADDs 6–7 address the second failure axis identified in our analysis: reward hacking, where the agent’s measured reward remains high while its true performance degrades. Unlike the reward deterioration axis covered by ADDs 2–5, where

the environment changes and the agent’s performance follows, reward hacking is an adversarial failure mode in which the agent itself exploits weaknesses in the reward specification, the evaluator, or the feedback loop. ADD 6 determines how to detect such exploitation, and ADD 7 determines how to mitigate it once detected or anticipated. The two decisions are closely coupled: as shown in Figure 2, ADD 6 and ADD 7 interact tensely, mitigations such as RLHF can induce context-dependent hiding that degrades *ChainOfThoughtAnalysis* in ADD 6, requiring architectural separation of detection and mitigation concerns. Furthermore, ADD 4 and ADD 6 complement each other, and ADD 6 limits ADD 4: reward signal tests in ADD 4 detect downward drift but miss upward gaming or reward inflation, meaning ADD 4 and ADD 6 must be combined for full coverage across both failure axes. A further asymmetry distinguishes this pair from ADDs 2–5: detection in ADD 6 is fundamentally harder than performance monitoring, because a reward-hacking agent may actively conceal its behavior from the monitoring infrastructure, meaning that the choice of mitigation architecture in ADD 7 must cater for the possibility that ADD 6 detectors are evaded.

ADDs 6–7 collectively yield 19 decision options and 25 decision drivers, reflecting the depth and diversity of the reward hacking threat landscape; the full catalog is available in our replication package [8]. Two dominant forces constrain the entire option space of both ADDs. *UnhackabilityHardness*, the recognition that no single detector or mitigation is sufficient against a capable agent, rules out any single-technique approach and mandates layered strategies across both decisions. *ScaleCompounding* reinforces this: as agent capabilities increase, hacking becomes subtler and harder to detect, pushing practitioners toward options that remain robust under capability growth rather than those optimized for current behavior. Within that constrained space, the differentiating forces resolve into two option families that span both ADDs. The first family, *Behavioral and Evaluator-Facing Strategies*, grouping options that observe or stress-test what the agent does: *MetricDivergenceDetection*, *AdversarialTesting*, *BehavioralTestingOnHeldOutEnvironments*, *EnsembleRewardModelAgreement*, and *AnomalyDetectionWithTrustedPolicy* in ADD 6, and *RewardFunctionRedesign*, *PeriodicRewardModelRefresh*, and *DecoupledApproval* in ADD 7, is driven by *GoodhartsLaw*, *SpecificationGaming*, and *EvaluatorHackability*, which together motivate cross-validating multiple signals and evaluators rather than relying on any single metric or judge. The second family, *Internal Reasoning and Structural Strategies*, grouping options that intervene at the level of the agent’s objectives or representations: *ChainOfThoughtAnalysis*, *VerbalisationSelfReport*, and *Deployment-TimeFeedbackLoopSimulation* in ADD 6, and *RegularisationToReferencePolicy*, *PreferenceAsReward*, *InoculationPrompting*, *MyopicOptimisation*, and *GatedRewardAccumulation* in ADD 7, is driven by *Covert-Misalignment*, *EvaluatorDeception*, and *InContextRewardHacking*, which motivate looking inside the agent’s reasoning and restructuring its optimization horizon rather than relying solely on external behavioral signals. *AlignmentImpermanence* and *RewardModelStaleness* apply across both families, favoring options

that treat alignment as an ongoing process, through *TargetedRLHF*, *PeriodicRewardModelRefresh*, and continued monitoring, rather than a property established once at training time.

5 Discussion

5.1 Discussion of the Research Questions

RQ1: We identified 7 ADDs, 57 decision options, and 72 decision drivers from the gray literature regarding monitoring deployed RL systems. The seven ADDs naturally organize into three layers, reflecting two distinct failure axes: reward deterioration and reward hacking. ADD 1 establishes the safe exploration envelope that any subsequent monitoring must operate within. ADDs 2–5 form the detection layer: ADD 2 feeds directly into ADD 3, as detection signals trigger automated responses; ADD 2 also constrains ADD 4, since the signal repertoire determines which statistical tests are applicable; and ADD 4 in turn constrains ADD 5, since the selected test determines which calibration methods are valid. ADD 4 and ADD 6 complement each other, and ADD 6 limits ADD 4, since reward signal tests detect downward drift but miss upward gaming, and the two must be combined for full coverage across both failure axes. ADDs 6 and 7 interact tensely: mitigations such as RLHF can induce context-dependent hiding, degrading detection capabilities in ADD 6, requiring architectural separation of detection and mitigation concerns.

RQ2: Several significant relations between decisions and their decision options emerged from the analysis. Several options recur across decisions, reflecting shared infrastructure that simultaneously unlocks choices in multiple ADDs. *KLDivergenceFromReferencePolicy* appears as a monitoring signal in ADD 2 and as a regularization technique in ADD 7, reflecting its dual role as both a diagnostic and a corrective instrument. *ShadowDeployment* appears as a response option in ADD 3 and as an evaluation mechanism supporting detection in ADD 6, connecting the detection and reward hacking layers at the option level. Within the detection layer, the signal repertoire chosen in ADD 2 constrains the statistical tests applicable in ADD 4: *Agent-Internal Signals* with episodic structure pair naturally with *CovarianceWeightedMean-Test* and *BootstrapFAR*, while *Environment-Facing Signals* motivate *AdaptiveThreshold* in ADD 5 since distributional shift invalidates fixed baselines. At the mitigation level, *VerbalisationSelfReport* in ADD 7 is only meaningful if *ChainOfThoughtAnalysis* has been adopted in ADD 6, and *PeriodicRewardModelRefresh* as a mitigation is most directly motivated when *Reward-ModelScoreMonitoring* has been selected in ADD 2.

RQ3: Our analysis revealed 72 decision drivers across the seven ADDs, which fall broadly into five categories. *Safety and alignment drivers*, including *Safety-Constraints*, *AlignmentImpermanence*, and *CovertMisalignment*, are the most pervasive category and appear across all three layers, reflecting the centrality of safe and aligned behavior as a non-negotiable constraint in RL deployment. *Environment and signal dynamics drivers*, including *EnvironmentNonStationarity*,

SignalNonStationarity, and *DriftInevitability*, motivate adaptive, multi-signal monitoring strategies and appear most prominently in the detection layer, collectively disfavoring static single-metric approaches. *Operational cost and reliability drivers*, including *CatastrophicDegradation*, *FlappingRisk*, and *FalseAlarmCost*, introduce opposing pressures most visible in ADD 3 and ADD 5, where the former demands fast decisive intervention and the latter two penalize overly sensitive triggers. *Specification and evaluation integrity drivers*, including *GoodhartsLaw*, *UnhackabilityHardness*, and *RewardModelStaleness*, are concentrated in ADDs 6 and 7, reflecting the adversarial character of the reward hacking axis. *Governance and traceability drivers*, including *GovernanceTraceability* and *SecurityCompliance*, cut across all layers, favoring monitoring architectures whose signals, thresholds, and interventions can be versioned and audited.

5.2 Threats to Validity

Our study applies Straussian GT [6] exclusively to gray literature, and several threats must be acknowledged. The risk of method slurring [31] is mitigated by adherence to all essential GT activities: simultaneous data collection and analysis, constant comparison, open, axial, and selective coding, and systematic memo writing. No seed categories or pre-existing frameworks were used to initiate coding. Interpretive bias is inherent to any qualitative methodology; we reduced this risk through iterative cross-checking between the two authors, with all coding decisions documented in memos and traceable to their sources in our replication package [8]. The gray literature corpus introduces a source-quality threat: RL monitoring is less mature than general ML deployment, and certain sub-domains, such as safety-critical robotics or regulated financial RL, may be underrepresented. This risk was mitigated by sampling to theoretical saturation. As with any GT study, we make no claim to capture all phenomena in the domain exhaustively or to quantify their frequency; the modeled decisions, options, and drivers represent observed practitioner concerns, not prescriptive recommendations [38]. The qualitative nature of our study introduces potential interpretive bias, which was mitigated through iterative cross-checking between all authors throughout the coding process. Any differences in interpretation were resolved through discussion and reaching a consensus.

5.3 Model Usage

The ADD model serves as a practical decision support tool for practitioners designing monitoring tools for deployed RL systems, providing a structured roadmap of seven decisions, from bounding exploration (ADD 1), through signal selection, statistical testing, and false alarm calibration (ADDs 2–5), to reward hacking detection and mitigation (ADDs 6–7), replacing ad hoc discovery with a systematic starting point.

6 Conclusions

We presented seven architectural design decisions (ADDs) for monitoring deployed RL systems, derived from a Straussian Grounded Theory study of gray literature. The ADDs organize along a layered monitoring pipeline addressing two distinct failure axes, reward deterioration and reward hacking, and collectively yield 57 decision options and 72 decision drivers. We formally modeled the relations among decisions, including directed dependencies within the detection layer and a tense interaction between reward-hacking detection and mitigation that requires architectural separation. The resulting ADD catalog, together with the UML-based overview and the full replication package [8], is intended to support practitioners in navigating the complex decision space of RL monitoring.

7 Data Availability

For replicability, all project artifacts, such as our GT memos with source URLs and our ADD model and model visualizations, are available in our replication package [8].

Acknowledgements. This research was funded by the FFG (Austrian Research Promotion Agency) project BEAM (no. FO999933314).

References

1. Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D.: Concrete problems in AI safety. arXiv preprint arXiv:1606.06565 (2016)
2. Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M.Y., Madry, A., Zaremba, W., Pachocki, J., Farhi, D.: Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv preprint arXiv:2503.11926 (2025)
3. Bosch, J., Olsson, H.H., Crnkovic, I.: Engineering AI Systems: A Research Agenda. Artificial Intelligence Paradigms for Smart Cyber-Physical Systems pp. 1–19 (2021)
4. Chen, S., Sun, Y., Li, D., Wang, Q., Hao, Q., Sifakis, J.: Runtime Safety Assurance for Learning-enabled Control of Autonomous Driving Vehicles. In: 2022 International Conference on Robotics and Automation (ICRA). pp. 8978–8984. IEEE (2022)
5. Corbin, J., Strauss, A.: Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory. Sage publications (2014)
6. Corbin, J.M., Strauss, A.: Grounded Theory Research: Procedures, Canons, and Evaluative Criteria. Qualitative Sociology **13**(1), 3–21 (1990)
7. Danesh, M.H., Fern, A.: Out-of-Distribution Dynamics Detection: RL-Relevant Benchmarks and Results. arXiv preprint arXiv:2107.04982 (2021)
8. Fang, Z., Warnett, S.J., Zdun, U.: Architectural Design Decisions for Monitoring Deployed Reinforcement Learning Systems (May 2026). <https://doi.org/10.5281/zenodo.20305497>
9. Fang, Z., Zdun, U.: Detecting Environment Drift in Reinforcement Learning using a Gaussian Process. In: 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI). pp. 992–999. IEEE (2024)

10. Fang, Z., Zdun, U.: EDSVM: A Novel Approach for Environment Drift Detection in Reinforcement Learning using Synthetic Drifted Examples. In: 28th European Conference on Artificial Intelligence (2025)
11. Ferreira, R.S., Guérin, J., Delmas, K., Guiochet, J., Waeselynck, H.: Safety Monitoring of Machine Learning Perception Functions: A Survey. *Computational Intelligence* **41**(2), e70032 (2025)
12. Garousi, V., Felderer, M., Mäntylä, M.V.: Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* **106**, 101–121 (2019)
13. Greenberg, I., Mannor, S.: Detecting Rewards Deterioration in Episodic Reinforcement Learning. In: International Conference on Machine Learning. pp. 3842–3853. PMLR (2021)
14. Greenblatt, R., Denison, C., Wright, B., Roger, F., MacDiarmid, M., Marks, S., Treutlein, J., Belonax, T., Chen, J., Duvenaud, D., et al.: Alignment faking in large language models. *arXiv preprint arXiv:2412.14093* (2024)
15. Haider, T., Roscher, K., Schmoeller da Roza, F., Günnemann, S.: Out-of-Distribution Detection for Reinforcement Learning Agents with Probabilistic Dynamics Models. In: Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems. pp. 851–859 (2023)
16. Karval, R., Singh, K.N.: Catching Silent Failures: A Machine Learning Model Monitoring and Explainability Survey. In: 2023 OITS International Conference on Information Technology (OCIT). pp. 526–532. IEEE (2023)
17. Laidlaw, C., Singhal, S., Dragan, A.: Correlated Proxies: A New Definition and Improved Mitigation for Reward Hacking. *arXiv preprint arXiv:2403.03185* (2024)
18. Leest, J., Raibulet, C., Lago, P., Gerostathopoulos, I.: From Tea Leaves to System Maps: A Survey and Framework on Context-aware Machine Learning Monitoring. *IEEE Transactions on Software Engineering* (2025)
19. Lwakatare, L.E., Raj, A., Bosch, J., Olsson, H.H., Crnkovic, I.: A taxonomy of software engineering challenges for machine learning systems: An empirical investigation. In: international conference on agile software development. pp. 227–243. Springer (2019)
20. Mallozzi, P., Castellano, E., Pelliccione, P., Schneider, G., Tei, K.: A Runtime Monitoring Framework to Enforce Invariants on Reinforcement Learning Agents Exploring Complex Environments. In: 2019 IEEE/ACM 2nd International Workshop on Robotics Software Engineering (RoSE). pp. 5–12. IEEE (2019)
21. Nascimento, E., Nguyen-Duc, A., Sundbø, I., Conte, T.: Software engineering for artificial intelligence and machine learning software: A systematic literature review. *arXiv preprint arXiv:2011.03751* (2020)
22. Naveed, H., Barnett, S., Arora, C., Grundy, J., Khalajzadeh, H., Haggag, O.: Monitoring machine learning systems: A multivocal literature review. *arXiv preprint arXiv:2509.14294* (2025)
23. Ntontos, E., Warnett, S.J., Zdun, U.: Supporting Architectural Decision Making on Training Strategies in Reinforcement Learning Architectures. In: 2024 IEEE 21st International Conference on Software Architecture (ICSA). pp. 90–100. IEEE (2024)
24. Pan, A., Bhatia, K., Steinhardt, J.: The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models. *arXiv preprint arXiv:2201.03544* (2022)
25. Protschky, D., Lämmermann, L., Hofmann, P., Urbach, N.: What Gets Measured Gets Improved: Monitoring Machine Learning Applications in Their Production Environments. *IEEE Access* (2025)

26. Rainer, A., Williams, A.: Using blog-like documents to investigate software practice: Benefits, challenges, and research directions. *Journal of Software: Evolution and Process* **31**(11), e2197 (2019)
27. Schröder, T., Schulz, M.: Monitoring machine learning models: a categorization of challenges and methods. *Data Science and Management* **5**(3), 105–116 (2022)
28. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D.: Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems* **28** (2015)
29. Serban, A., Van der Blom, K., Hoos, H., Visser, J.: Adoption and effects of software engineering best practices in machine learning. In: *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. pp. 1–12 (2020)
30. Skalse, J., Howe, N., Krashennnikov, D., Krueger, D.: Defining and Characterizing Reward Gaming. *Advances in Neural Information Processing Systems* **35**, 9460–9471 (2022)
31. Stol, K.J., Ralph, P., Fitzgerald, B.: Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In: *Proceedings of the 38th International Conference on Software Engineering*. pp. 120–131 (2016)
32. Sutton, R.S., Barto, A.G., et al.: *Reinforcement Learning: An Introduction*. MIT Press, Cambridge (1998)
33. Wang, C., Erfani, S., Alpcan, T., Leckie, C.: Oil-AD: An Anomaly Detection Framework for Sequential Decision Sequences. *arXiv preprint arXiv:2402.04567* (2024)
34. Wang, Z., Chen, C., Li, H.X., Dong, D., Tarn, T.J.: Incremental Reinforcement Learning with Prioritized Sweeping for Dynamic Environments. *IEEE/ASME Transactions on Mechatronics* **24**(2), 621–632 (2019)
35. Warnett, S.J., Zdun, U.: Architectural Design Decisions for Machine Learning Deployment. In: *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. pp. 90–100. IEEE (2022)
36. Warnett, S.J., Zdun, U.: Architectural Design Decisions for the Machine Learning Workflow. *Computer* **55**(3), 40–51 (2022)
37. Washizaki, H., Uchida, H., Khomh, F., Guéhéneuc, Y.G.: Studying Software Engineering Patterns for Designing Machine Learning Systems. In: *2019 10th International Workshop on Empirical Software Engineering in Practice (IWESEP)*. pp. 49–495. IEEE (2019)
38. Zieris, F., Prechelt, L.: On knowledge transfer skill in pair programming. In: *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. pp. 1–10 (2014)
39. Zolfagharian, A., Abdellatif, M., Briand, L.C., et al.: SMARLA: A Safety Monitoring Approach for Deep Reinforcement Learning Agents. *IEEE Transactions on Software Engineering* **51**(1), 82–105 (2024)